Why GPUs Lose at Zone-Graph Exploration: Ten Measured Attempts on Apple Silicon

Abstract

Symbolic reachability for timed automata generates millions of independent successor computations, which makes it look like an ideal GPU workload. We report that it is not, and — more usefully — we identify the mechanism. We implemented GPU successor generation and per-state zone merging in Metal, in *three* different DBM representations and across ten measured configurations, and compared each against an optimised 12-core CPU implementation of the same algorithm. Every GPU component is slower than its CPU counterpart: expansion 1400 ms versus 491 ms, merging 258–571 ms versus 100 ms; an adaptive router given a free per-level choice selects the CPU on 122 of 124 levels. We refute the two hypotheses a practitioner would reach for first. Branch divergence is *not* the cause: 100.0% of lanes survive to do full work on Viking and 71.5% on Fischer, so stream compaction would recover nothing. Instead the cause is a *working-set dilemma*: a DBM is too large to hold in registers and too large to hold on-chip at useful occupancy. We further show that no change of representation escapes this, because one-zone-per-thread already extracts all available parallelism, and any cooperative mapping must beat a 32× deficit with at most $dim\times$ intra-zone parallelism. We close by scoping the result: the dominant cause is a threadgroup-memory capacity limit that datacenter GPUs relax by 7×, so the honest claim is about unified-memory Apple GPUs, not about GPUs in general.

1 Introduction

Zone-graph reachability appears tailor-made for a GPU. A BFS level contains millions of $(\text{state}, \text{edge})$ pairs; each is expanded independently; the arithmetic is small dense integer matrix work. Yet across a sustained implementation effort we were unable to make a GPU implementation beat a 12-core CPU implementation of the same algorithm on the same chip, and the reasons turned out to be interesting enough to report.

This paper is a negative result with a mechanism. Its value is threefold. First, it refutes, *on measurement rather than intuition*, the hypothesis most practitioners (including the authors) reach for first — branch divergence — and thereby saves the reader from implementing the standard fix. Second, it identifies the actual cause and shows it is not a coding failure but a consequence of the algorithm's shape. Third, it delimits the claim precisely: the dominant cause is a hardware capacity that a datacenter GPU relaxes, so the result should not be read as "GPUs cannot do this".

Contributions.

- Three GPU DBM representations, all implemented and validated against exact Uppaal state counts (Sect. 2.1).
- An adaptive per-level CPU/GPU merge router, whose selections are themselves a measurement instrument (Sect. 2.2).
- A refutation of the divergence hypothesis and identification of the working-set dilemma (Sect. 4), plus a counting argument showing no representation escapes it (Sect. 4.4).
- A scoped outlook for datacenter GPUs (Sect. 5).

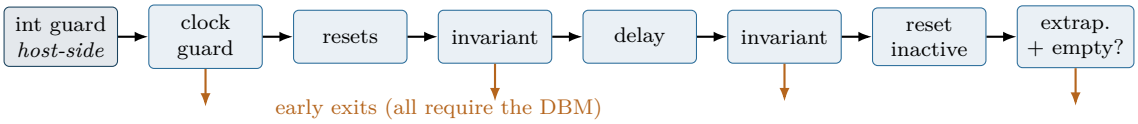


Figure 1: Per-work-item successor pipeline. Only the first stage is DBM-free, and it already runs on the host, so a cheap GPU pre-filter has nothing left to reject.

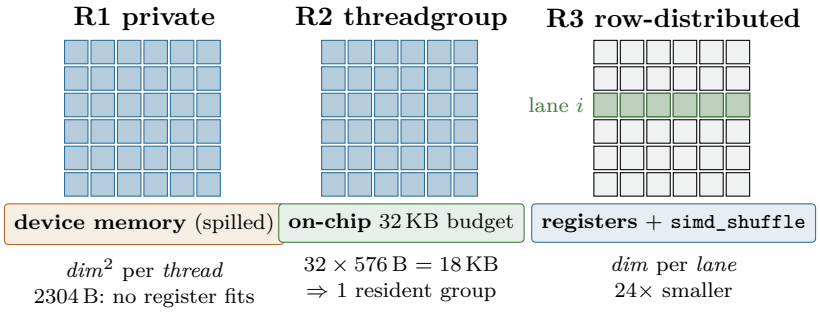


Figure 2: Three DBM representations. R3 solves the footprint problem outright, and still loses — for the reason given in Sect. 4.4.

2 Background and GPU Design

A symbolic state pairs a discrete location vector with a zone stored as a $(m{+}1) \times (m{+}1)$ Difference Bound Matrix in canonical form. The successor pipeline for one (state, edge) pair is shown in Fig. 1: integer guard, clock guard, resets, combined invariant, delay, invariant again, inactive-clock reset, extrapolation, emptiness. Integer guards are evaluated *host-side* while the work list is built, so the kernel receives only pairs whose discrete part is already known feasible — a fact that matters in Sect. 4.

2.1 Three DBM representations

We implemented three mappings of a zone onto the GPU (Fig. 2), all producing identical, UPPAAL-exact state counts.

R1: private dense matrix (baseline). One work item per thread; the zone lives in a per-thread array `int z[MAXDIM*MAXDIM]`. At $MAXDIM = 24$ this is 2304 B.

R2: threadgroup-resident dense matrix. One work item per thread, but the zone lives in threadgroup (on-chip) memory, sized by the model's real *dim* rather than *MAXDIM*, with an odd stride to avoid bank conflicts.

R3: row-distributed. One work item per *SIMD group*; lane i owns row i of the matrix in registers. Per-lane footprint falls from dim^2 to dim — a 24× reduction. Floyd–Warshall obtains $d[k][\cdot]$ by `simd_shuffle`, register-to-register; *emptiness* folds through `simd_any`.

2.2 Adaptive CPU/GPU merge routing

Per-discrete-state merging ($\preceq_{LU}$ subsumption plus convex union) is a second kernel: one bucket per thread. Because bucket shape varies enormously between models, we route each BFS level to whichever device is predicted faster. The router calibrates the throughput of each path on early levels — measuring $work/ms$ with $work = \sum_b |b|^2$, the natural cost model for pairwise antichain folding — then routes each later level to the higher-throughput device, updating its estimate with an exponential moving average (Fig. 3).

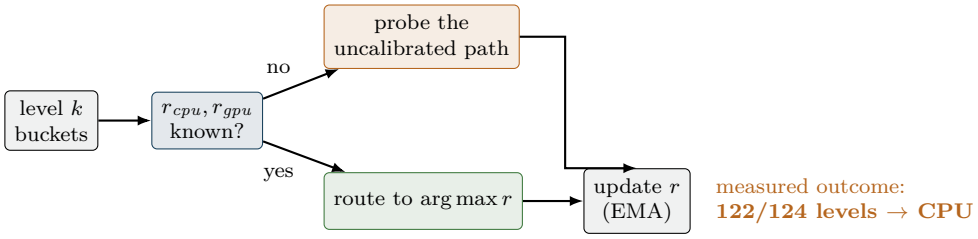


Figure 3: Adaptive per-level merge routing. Used as a measurement instrument, it selects the GPU only for the single calibration probe on each model.

Table 1: GPU components against their CPU counterparts. Every row favours the CPU.

Operation	GPU	CPU	Note
Expansion (Viking 15)	1400 ms	**491 ms**	CPU 2.85× faster
Merge, zones written by GPU	258 ms	**100 ms**	GPU's best case
Merge, zones written by CPU	571 ms	**100 ms**	coherency penalty
Adaptive router's free choice	2 levels	**122 levels**	of 124 total

The router is useful in its own right, but its scientific value here is as an *instrument*: given a free choice on every level, its selections are an unbiased verdict on which device is actually faster.

3 Measurements

All numbers are on an Apple M4 Pro: 8 performance + 4 efficiency CPU cores, a 16-core GPU (≈2000 shader lanes), 24 GB unified memory. Models are Viking N=15 ($m{+}1 = 12$, so a 12×12 DBM = 576 B; 8 355 840 successors) and Fischer N=10.

A consequence worth stating plainly: the configuration that is fastest on Fischer nominally uses the Metal backend, but performs 100% of its work on the CPU. Its advantage is *algorithmic* — a bucketed, batched per-state merge over compact arrays — and removing the GPU from that configuration makes it faster still (0.59 s → 0.42 s). It must not be reported as GPU acceleration.

4 Why: two refuted hypotheses and one real cause

4.1 Divergence is not the cause

The standard explanation is lockstep execution: lanes run in groups of 32, most edges fail their guard early, and one survivor drags 31 idle lanes through the remaining pipeline. We instrumented lane survival directly:

Model	Work items	Survivors	Survive
Viking N=15	8 355 840	8 355 840	**100.0%**
Fischer N=10	3 317 950	2 372 190	71.5%

On Viking there is *no* early-exit divergence whatsoever. Stream compaction or wavefront scheduling — the standard remedy, and the idea we were about to implement — would recover 0% there and at most 28% on Fischer. Moreover the only DBM-free filter, the integer guard, is already applied host-side (Fig. 1), so a cheap pre-pass has nothing to reject. *We report this specifically so that others do not implement it.*

Table 2: Expansion time by representation. R3 cuts per-lane footprint 24× and is the slowest of the three.

	Representation	Viking N=15	Fischer N=10
R1	private dense (spills to device memory)	**1425 ms**	**183 ms**
R2	threadgroup-resident dense	1587 ms	277 ms
R2′	+ bank-conflict padding	1575 ms	297 ms
R3	row-distributed across SIMD lanes	2536 ms	351 ms

4.2 Arithmetic intensity is not (yet) the cause

Each successor moves ≈1152 B for ≈2600 integer operations: 2.3 ops/byte against a roofline ridge point of ≈10, so the kernel is memory-bound *in character*. But the achieved rates are nowhere near that bound:

	Rate	% compute peak	% bandwidth
GPU expand (1400 ms)	5.97 M succ/s	**0.5%**	2.5%
CPU expand (491 ms)	17.0 M succ/s	≈20% (of CPU)	7%

The bandwidth ceiling for this kernel is ≈237 M succ/s, roughly 40× above what the GPU achieves. The GPU is limited by *utilisation*, not by a physical roof: the CPU runs at ≈20% of its peak while the GPU runs at 0.5% of its own.

4.3 The working-set dilemma

Because Floyd–Warshall's k loop is strictly sequential, there is almost no parallelism *inside* one zone, so every thread must own a whole zone. That creates a dilemma with no good side, and we measured both horns (Table 2).

Keeping the zone **private** means 2304 B per thread, far beyond the ≈128–256 registers a thread receives, so it spills to thread-local *device* memory and all ≈2600 DBM operations per successor run at device-memory latency. Keeping it **on-chip** costs $32 \times 576\,\text{B} = 18\,\text{KB}$ of Apple's 32 KB threadgroup budget, leaving roughly one resident threadgroup per core: occupancy collapses and there is nothing left with which to hide latency. R1 wins because sheer thread count buys more latency hiding than on-chip residency buys in access speed.

4.4 No representation escapes it

R3 was designed to break the dilemma and does so on its own terms — *dim* ints per lane, comfortably register-resident — yet it is the slowest. The reason is a counting argument, and it generalises to *any* cooperative mapping.

One zone per *thread* keeps 32 independent zones in flight per SIMD group. Any scheme that cooperates *within* a zone — R2's shared tile or R3's distributed rows — processes 1 zone per SIMD group. Cooperation must therefore make a single zone more than 32× faster merely to break even. The parallelism available inside one zone is only *dim*-wide: at $dim = 12$ that is 12 of 32 lanes busy, an immediate 2.7× loss, with a ceiling of 12× before paying for `simd_shuffle` traffic and the serial k loop. **Twelve-fold best case against a thirty-two-fold deficit.** Both cooperative attempts lost, by different mechanisms, exactly as predicted.

This is the deepest reason a "GPU-friendly representation" does not exist here: the workload's parallelism lives *across* zones and is already fully exploited by the simplest possible mapping. The dense matrix inherited from UPPAAL is not what holds the GPU back.

Table 3: The dominant cause is substantially an Apple-Silicon capacity constraint.

	Apple M4 Pro	NVIDIA H100
Shared / threadgroup memory	32 KB per group	228 KB per SM
Memory bandwidth	273 GB/s (shared with CPU)	≈3.3 TB/s dedicated
Lanes	≈2 000	≈16 900

4.5 Unified-memory coherency

Finally, a hybrid in which the CPU expands and the GPU merges — attractive because the two phases favour different hardware — is defeated by cache coherency. Identical merge work costs 258 ms when the zones were written by the GPU and 571 ms when written by the CPU, because the GPU must snoop them out of CPU caches. The moment the CPU produces the zones, the GPU is the wrong consumer.

5 Outlook: would a larger GPU change the answer?

Partly, and the claim should be scoped accordingly. The dominant cause, Sect. 4.3, is a *capacity* limit, and datacenter GPUs relax it substantially (Table 3). With 228 KB of shared memory per SM one could hold ≈390 zones on-chip, so R2's failure mode largely disappears. Because the workload is latency-bound with millions of *independent* zones, absolute throughput should also scale with GPU size even at unchanged utilisation; scaling crudely by lanes and bandwidth suggests 50–70 M succ/s against our CPU's 17 M.

What would *not* improve: the serial k loop, and the counting argument of Sect. 4.4 (warps are also 32 lanes wide), so one-zone-per-thread remains the right mapping — merely a wider one. Coherency becomes *worse* on a discrete GPU, since PCIe transfers replace cache snooping unless the entire search is made GPU-resident, which in turn requires a correct lock-free device-side hash table; our attempt at one produced incorrect results because Metal Shading Language exposes only relaxed atomics, with no acquire/release pairing for the publication step.

Two caveats for a fair follow-up: an H100 should be compared against a similarly priced CPU (a 128-core server part, not 12 consumer cores), and GPU residency is a prerequisite rather than an optimisation.

6 Conclusion

Across ten measured configurations and three DBM representations, GPU zone-graph exploration on a unified-memory Apple GPU is slower than an optimised CPU implementation of the same algorithm, component by component. The cause is neither branch divergence (refuted: 100% lane survival) nor arithmetic intensity (refuted: 0.5% of peak, 40× from the roof), but a working-set dilemma in which a DBM is simultaneously too large for registers and too large for on-chip residency at useful occupancy — and a counting argument that forecloses the cooperative mappings that would otherwise escape it. We scope the result to unified-memory GPUs with small threadgroup budgets, and identify the shared-memory capacity of datacenter parts as the specific quantity that would have to change.

References

[1] Dalsgaard, A.E., Laarman, A., Larsen, K.G., Olesen, M.C., van de Pol, J.: Multi-core reachability for timed automata. In: FORMATS. LNCS, vol. 7595, pp. 91–106. Springer (2012)

[2] Behrmann, G., David, A., Larsen, K.G.: A tutorial on Uppaal. In: SFM. LNCS, vol. 3185, pp. 200–236. Springer (2004)

[3] Behrmann, G., Bouyer, P., Larsen, K.G., Pelánek, R.: Lower and upper bounds in zone-based abstractions of timed automata. STTT **8**(3), 204–215 (2006)

[4] Laine, S., Karras, T., Aila, T.: Megakernels considered harmful: Wavefront path tracing on GPUs. In: High-Performance Graphics, pp. 137–143 (2013)

[5] Williams, S., Waterman, A., Patterson, D.: Roofline: An insightful visual performance model for multicore architectures. Commun. ACM **52**(4), 65–76 (2009)